

Competitive Programming

Section 7 Editorial: Foundation mixed contest

Competitive Programming Course

2026-07-04

Table of contents

1	How to read this editorial	3
2	Problem A: Inventory Cleanup	4
2.1	Restatement	4
2.2	Important constraints and traps	4
2.3	Observations	4
2.4	Sample walk-through	4
2.5	Algorithm	5
2.6	Correctness	5
2.7	Complexity	5
2.8	C++ solution	5
2.9	Python solution	6
2.10	Common wrong answers	6
3	Problem B: Peak Load	7
3.1	Restatement	7
3.2	Important constraints and traps	7
3.3	Observations	7
3.4	Sample walk-through	7
3.5	Algorithm	8
3.6	Correctness	8
3.7	Complexity	8
3.8	C++ solution	8
3.9	Python solution	9
3.10	Common wrong answers	10
4	Problem C: Best Study Streak	11
4.1	Restatement	11
4.2	Important constraints and traps	11
4.3	Observations	11
4.4	Sample walk-through	11
4.5	Algorithm	11
4.6	Correctness	12
4.7	Complexity	12
4.8	C++ solution	12
4.9	Python solution	13
4.10	Common wrong answers	14
5	Contest testing strategy	15
6	What to carry forward	16

1 How to read this editorial

This is a mixed-contest editorial. For each problem, first identify the clue that selects the technique. Then read the proof and implementation details.

The goal is not only to know the solution after seeing it. The goal is to notice why this solution should have been considered during the contest.

2 Problem A: Inventory Cleanup

2.1 Restatement

Delete the fewest items so that at most k distinct colors remain.

2.2 Important constraints and traps

Colors can be large or negative, so use a map/dictionary. If $k = 0$, all items must be deleted. If the number of distinct colors is already at most k , the answer is zero.

The actual color labels do not matter after counting. Only frequencies matter.

2.3 Observations

To reduce the distinct-color count, deleting only some items of a color is not useful unless the whole color disappears. A color remains distinct as long as at least one item of that color remains.

Therefore each deleted color group costs its frequency. If there are d distinct colors and we need at most k , then we must remove:

```
max(0, d - k)
```

entire color groups. To minimize deletions, remove the groups with smallest frequencies.

2.4 Sample walk-through

Colors:

```
1 1 2 2 3 4 4
```

Frequencies:

color	count
1	2
2	2
3	1
4	2

There are 4 distinct colors. If $k = 2$, remove 2 color groups. The smallest group sizes are 1 and 2, so the answer is 3.

2.5 Algorithm

```
count color frequencies
sort the frequency values ascending
remove_groups = max(0, distinct_count - k)
answer = sum of the remove_groups smallest frequencies
print answer
```

2.6 Correctness

Any solution that leaves at most k colors must completely delete at least $d - k$ color groups, where d is the number of distinct colors. Deleting only part of a color group does not reduce the distinct count and can only increase the number of deletions, so an optimal solution deletes whole groups.

Each deleted group has cost equal to its frequency. Among all choices of $d - k$ groups, the minimum total cost is obtained by choosing the smallest $d - k$ frequencies. The algorithm does exactly that after sorting. Therefore the printed number is minimum.

2.7 Complexity

Counting is $O(n)$ expected time. Sorting the distinct frequencies is $O(d \log d)$, where $d \leq n$. Memory is $O(d)$.

2.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, k;
        cin >> n >> k;
        unordered_map<int, int> freq;
        freq.reserve(2 * n + 1);
        for (int i = 0; i < n; ++i) {
            int color;
            cin >> color;
            ++freq[color];
        }
    }
}
```

```

vector<int> counts;
for (auto [color, count] : freq) {
    counts.push_back(count);
}
sort(counts.begin(), counts.end());
int remove_groups = max(0, static_cast<int>(counts.size()) - k);
int answer = 0;
for (int i = 0; i < remove_groups; ++i) {
    answer += counts[i];
}
cout << answer << '\n';
}
return 0;
}

```

2.9 Python solution

```

from collections import Counter
import sys

def min_deletions(values: list[int], k: int) -> int:
    freq = sorted(Counter(values).values())
    remove = max(0, len(freq) - k)
    return sum(freq[:remove])

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    k = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    out.append(str(min_deletions(values, k)))
sys.stdout.write("\n".join(out))

```

2.10 Common wrong answers

- Deleting individual rare items without removing whole colors.
- Keeping the most frequent k colors but computing the deleted count incorrectly.
- Mishandling k = 0.
- Using a frequency array for large or negative color values.

3 Problem B: Peak Load

3.1 Restatement

Each job adds load w to every integer time in $[1, r]$. Find the maximum total load and the earliest time when it occurs.

3.2 Important constraints and traps

The sum of $n + m$ is bounded, so an $O(n + m)$ difference-array solution is intended. Loads may be negative, so initialize the best load carefully; it may also be negative.

The tie rule is earliest time. A left-to-right scan should update only on strictly greater load.

3.3 Observations

This is a range-add problem. For each job:

```
diff[1] += w
diff[r + 1] -= w
```

Then scan times from 1 to m , maintaining the current load.

3.4 Sample walk-through

Jobs:

```
[1,3] +5
[2,5] +4
[6,6] +10
```

Loads:

time	load
1	5
2	9
3	9
4	4
5	4
6	10

The maximum is 10 at time 6.

3.5 Algorithm

```
diff = zeros of length m + 2
for each job l, r, w:
    diff[l] += w
    diff[r + 1] -= w
cur = 0
best = -infinity
best_time = 1
for time from 1 to m:
    cur += diff[time]
    if cur > best:
        best = cur
        best_time = time
print best, best_time
```

3.6 Correctness

For a single job $[l, r, w]$, the running sum of `diff` includes w at every time t with $l \leq t \leq r$, and includes zero net contribution outside that range. By linearity of addition, after all jobs are marked, the running sum at time t equals the total load of all jobs covering t .

The scan visits times in increasing order. Whenever it sees a load strictly larger than all previous loads, it records that load and time. If the same maximum appears later, the scan does not update, so the earliest time remains stored. Therefore the output is the maximum load and its earliest occurrence.

3.7 Complexity

Each job is processed in $O(1)$, and the scan is $O(m)$. Memory is $O(m)$.

3.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
```

```

vector<long long> diff(m + 2, 0);
for (int i = 0; i < n; ++i) {
    int l, r;
    long long w;
    cin >> l >> r >> w;
    diff[l] += w;
    diff[r + 1] -= w;
}
long long cur = 0;
long long best = LLONG_MIN;
int best_time = 1;
for (int time = 1; time <= m; ++time) {
    cur += diff[time];
    if (cur > best) {
        best = cur;
        best_time = time;
    }
}
cout << best << ' ' << best_time << '\n';
}
return 0;
}

```

3.9 Python solution

```

import sys

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    m = data[idx + 1]
    idx += 2
    diff = [0] * (m + 2)
    for _ in range(n):
        l = data[idx]
        r = data[idx + 1]
        w = data[idx + 2]
        idx += 3
        diff[l] += w
        diff[r + 1] -= w
    cur = 0
    best = None
    best_time = 1
    for time in range(1, m + 1):
        cur += diff[time]

```

```
    if best is None or cur > best:
        best = cur
        best_time = time
    out.append(f"{best} {best_time}")
sys.stdout.write("\n".join(out))
```

3.10 Common wrong answers

- Updating on $\geq$ and returning the latest maximum instead of earliest.
- Initializing best to 0 when all loads can be negative.
- Forgetting the $r + 1$ marker.
- Applying every job to every time directly.

4 Problem C: Best Study Streak

4.1 Restatement

Find the longest contiguous subarray with sum at most k in a nonnegative array. If several longest streaks exist, print the earliest 1-indexed start. If no nonempty streak is valid, print 0.

4.2 Important constraints and traps

Values are nonnegative, so sliding window applies. The tie rule requires start tracking. The answer may be zero if every value is greater than k .

4.3 Observations

Use the same window as Section 6. After shrinking, `[left, right]` is the longest valid window ending at `right`. Compare it with the best answer so far.

Because `right` increases from left to right, equal-length windows often appear in start order, but the safest implementation still writes the tie comparison explicitly.

4.4 Sample walk-through

Values:

```
5 1 1 1 1 5
```

Limit 5.

The best streak is:

```
1 1 1 1
```

starting at position 2, length 4. The single 5 values are valid but shorter.

4.5 Algorithm

```

left = 0
sum = 0
best_len = 0
best_start = 0
for right from 0 to n - 1:
    sum += a[right]
    while sum > k:
        sum -= a[left]
        left += 1
    length = right - left + 1
    start = left + 1
    if length is better than current answer:
        update answer
print best_len, best_start

```

4.6 Correctness

After shrinking for a fixed **right**, the current window has sum at most **k**. Because all values are nonnegative, any window ending at **right** and starting before **left** has sum greater than **k**; otherwise the shrink loop would have stopped earlier. Thus the current window is the longest valid window ending at **right**.

The algorithm compares this best window for each right endpoint against the global best, using length first and start position second. Every valid window has some right endpoint, and for that endpoint it is no longer than the window considered by the algorithm. Therefore the final stored answer is the maximum length, with earliest start among ties.

If no nonempty valid window exists, **best_len** remains 0 and the algorithm prints 0 0, as required.

4.7 Complexity

Both pointers move only forward. Time is $O(n)$, memory is $O(1)$.

4.8 C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;

```

```

    long long limit;
    cin >> n >> limit;
    vector<long long> values(n);
    for (long long &value : values) cin >> value;

    int left = 0;
    long long total = 0;
    int best_len = 0;
    int best_start = 0;
    for (int right = 0; right < n; ++right) {
        total += values[right];
        while (left <= right && total > limit) {
            total -= values[left];
            ++left;
        }
        int length = right - left + 1;
        int start = left + 1;
        if (length > best_len || (length == best_len && length > 0 && start < best_start)) {
            best_len = length;
            best_start = start;
        }
    }
    cout << best_len << ' ' << best_start << '\n';
}
return 0;
}

```

4.9 Python solution

```

import sys

def solve(values: list[int], limit: int) -> tuple[int, int]:
    left = 0
    total = 0
    best_len = 0
    best_start = 0
    for right, value in enumerate(values):
        total += value
        while left <= right and total > limit:
            total -= values[left]
            left += 1
        length = right - left + 1
        start = left + 1
        if length > best_len or (length == best_len and length > 0 and start < best_start):
            best_len = length
            best_start = start
    return best_len, best_start

```

```

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    limit = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    length, start = solve(values, limit)
    out.append(f"{length} {start}")
sys.stdout.write("\n".join(out))

```

4.10 Common wrong answers

- Printing the end position instead of the start.
- Forgetting 0 0 when no nonempty window is valid.
- Applying the method to arrays with negative values.
- Updating the answer before shrinking.

5 Contest testing strategy

For a mixed set, each problem should get a different oracle:

- Problem A: brute force over which distinct colors to keep for very small distinct counts, or compare against sorted frequencies.
- Problem B: direct time simulation for small m .
- Problem C: quadratic enumeration of all subarrays.

The local random tests use these simple references. This is not extra polish; it is how you keep contest code honest.

6 What to carry forward

Phase 0 is now complete. The foundation tools are useful only if you can select them under time pressure. The next phase begins with binary search on the answer, where the same habits continue: read constraints, identify monotonicity, prove the predicate, and test edge cases.